

T.C
KONYA TEKNİK ÜNİVERSİTESİ
MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ
3. SINIF BAHAR DÖNEMİ
BİLİŞİM TEKNOLOJİLERİ UYGULAMASI ARASINAV/FİNAL(BÜT) RAPORU

Öğrencinin Adı-Soyadı: Alperen ÇUKUROVA-Muhammet Fatih YILMAZ
Numarası: 221220079-221220047
Danışman Adı Soyadı: Ersin KAYA
Sınav Tarihi: 29.05.2025
Projenin Adı: Kampüs Kantin

DÖNEM İÇİ YAPILAN ÇALIŞMALARIN ÖZETİ

- **Projenin Amacı:**
 - KampusKantin projesi, ilkokul ve ortaokul öğrencilerinin kantin alışverişlerini güvenli ve kontrollü bir şekilde yapabilmelerini sağlayan bir dijital çözümdür. Bu proje, iki temel soruna odaklanmaktadır:
 - Güvenlik ve Kontrol: Öğrencilerin nakit para taşımalarını ortadan kaldırarak, para kaybı ve dolandırılma riskini minimize eder. Her öğrenciye özel bir kart verilerek, sadece kendi kartları üzerinden işlem yapabilmeleri sağlanır.
 - Şeffaflık ve Takip: Veliler, çocuklarının kantin harcamalarını gerçek zamanlı olarak takip edebilir. Hangi ürünleri aldığı, ne kadar harcama yaptığı ve bakiye durumu gibi bilgilere anında erişim sağlanır. Bu sayede veliler, çocuklarının beslenme alışkanlıklarını kontrol edebilir ve gerektiğinde müdahale edebilir.
- **Teknik Altyapı:**
 - **Frontend Teknolojileri**
 1. **React Native (Mobil Uygulama)**
 - Velilerin çocuklarının harcamalarını takip edebileceği mobil uygulama
 - Profil yönetimi, bakiye yükleme, şifre değiştirme gibi temel işlemler
 - Kart kaybolduğunda bildirim yapabilme
 - Hesap hareketlerini görüntüleme

2. React (Web Uygulaması)

- Velilerin çocuklarının harcamalarını takip edebileceği web arayüzü
- Profil yönetimi, bakiye yükleme, şifre değiştirme gibi temel işlemler
- Kart kaybolduğunda bildirim yapabilme
- Hesap hareketlerini görüntüleme

○ Backend Teknolojileri

1. Node.js

- Kullanıcı işlemleri (kayıt, giriş, profil)
- Kart yönetimi (kart kontrolü, dondurma)
- Bakiye işlemleri (yükleme, sorgulama)
- Hesap hareketleri takibi

○ Veritabanı

1. MSSQL (Microsoft SQL Server)

- Kullanıcılar tablosu (öğrenci bilgileri, bakiye)
- Kartlar tablosu (kart durumları)
- HesapHareketleri tablosu (alışveriş kayıtları)

○ API Yapısı

1. Temel Endpoint'ler

- /api/kullanici/kayit - Yeni öğrenci kaydı
- /api/kullanici/giris - Öğrenci girişi
- /api/kullanici/profil - Profil bilgileri
- /api/kullanici/sifre-guncelle - Şifre değiştirme
- /api/kullanici/kartKontrol - Kart kontrolü
- /api/kullanici/kayıpKart - Kayıp kart bildirimi
- /api/kullanici/bakiyeYukle - Bakiye yükleme
- /api/kullanici/islemGecmisi - İşlem geçmişi

○ ESP32 ve PN532 Yapısı

- Kantin sisteminde kullanılan ESP32 mikrodenetleyicisi ile PN532 NFC kart okuyucu modülünün entegrasyonunu sağlar.
- Öğrenci, kartını okuyucuya okuttuğunda kartın ID'si alınır ve WiFi üzerinden sunucuya (backend API) gönderilir.
- Sunucu, kartın geçerliliğini ve bakiye durumunu kontrol eder ve sonucu ESP32'ye iletir. Böylece öğrencinin kartı ile güvenli ve hızlı bir şekilde işlem yapılması sağlanır.

○ Güvenlik Özellikleri

- Şifre güvenliği (minimum 6 karakter)
- Kart güvenliği (dondurma, kontrol)
- Oturum yönetimi
- Veri doğrulama

○ Veri Akışı

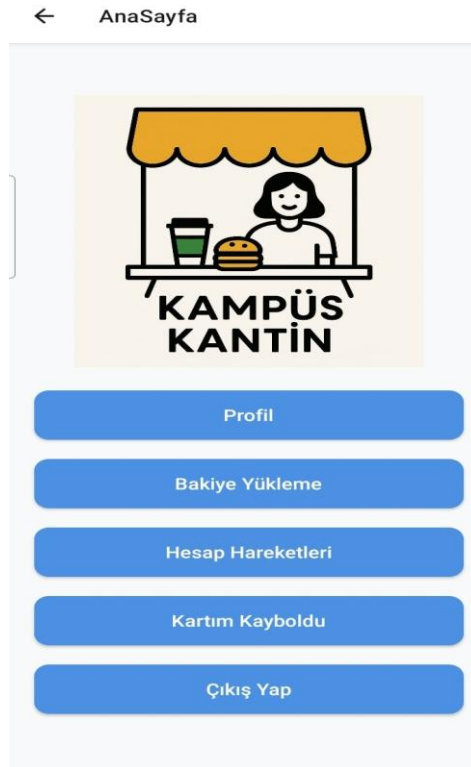
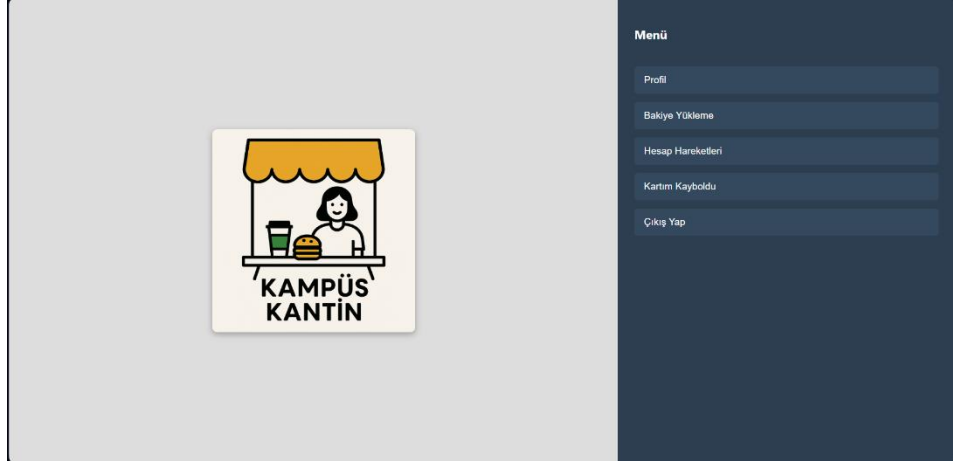
- Öğrenci kantinde alışveriş yaptığında:
 1. Kart kontrolü yapılır
 2. Bakiye kontrolü yapılır
 3. İşlem kaydedilir
 4. Bakiye güncellenir
 5. Veli bilgilendirilir

Bu yapı, öğrencilerin güvenli bir şekilde kantin işlemlerini yapabilmelerini ve velilerin bu işlemleri takip edebilmelerini sağlamaktadır. Sistem, basit ve kullanıcı dostu bir arayüz sunarken, gerekli tüm güvenlik önlemlerini de içermektedir.

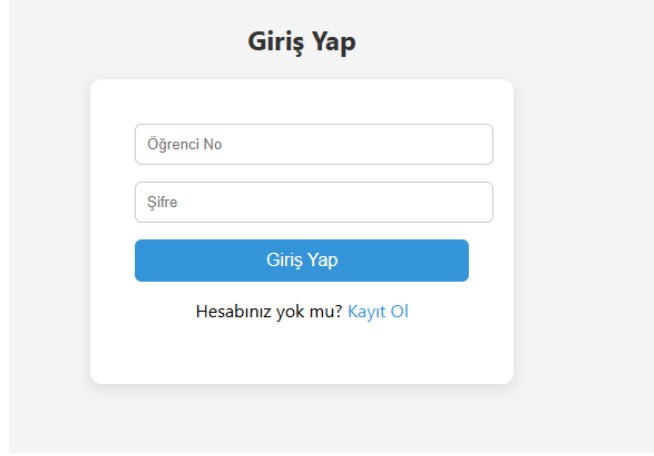
- **Sistem Mimarisi:**

- **Uygulama Ekranları**

- **Ana Sayfa (Home):** Kullanıcı sisteme giriş yaptıktan sonra karşısına çıkan ana menü ekranıdır. Buradan profil, bakiye yükleme, hesap hareketleri, kart kayboldu işlemleri ve çıkış yapma gibi temel işlemlere hızlıca erişim sağlanır. Hem webde hem mobilde aynı mantıkla çalışır.



- **Giriş Ekranı (Giriş Yap / Login):** Kullanıcıların (öğrenci veya veli) sisteme giriş yapmasını sağlayan ekrandır. Öğrenci numarası ve şifre ile giriş yapılır. Hesabı olmayanlar için kayıt olma yönlendirmesi de mevcuttur.



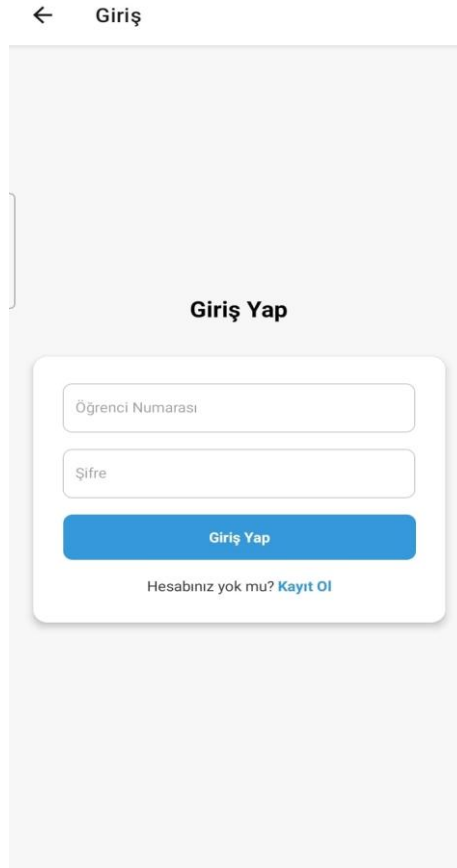
Giriş Yap

Öğrenci No

Şifre

Giriş Yap

Hesabınız yok mu? [Kayıt Ol](#)



← Giriş

Giriş Yap

Öğrenci Numarası

Şifre

Giriş Yap

Hesabınız yok mu? [Kayıt Ol](#)

- **Kayıt Ol Ekranı (Kayıt Ol/Register):** Yeni kullanıcıların sisteme kayıt olmasını sağlayan ekrandır. Kart ID, öğrenci numarası, ad, soyad ve şifre bilgileri girilerek kayıt işlemi tamamlanır. Şifre güvenliği ve kart geçerliliği kontrol edilir.

Kayıt Ol

Kart ID

Öğrenci Numarası

Ad

Soyad

Şifre

Şifre Tekrarı

Kaydol

← Kayıt

Kayıt Ol

Kart ID

Öğrenci Numarası

Ad

Soyad

Şifre

Şifre Tekrarı

Kaydol

- **Profil Ekranı (Profil Bilgileri/Profile):** Kullanıcının kişisel bilgilerini (Kart ID, Ad, Soyad, Öğrenci No, Bakiye) görüntüleyebildiği ve şifresini güncelleyebildiği ekrandır. Hem web hem mobilde benzer şekilde çalışır.

Profil Bilgileri

ID Kart ID: 12345678
Ad: Alper
Soyad: Çukurova
Öğrenci No: 221220079
Bakiye: 18.00 ₺

Şifre Güncelle

Mevcut Şifre:

Yeni Şifre:

Yeni Şifre (Tekrar):

Şifreyi Güncelle

← Profil

Profil Bilgileri

Kart ID: 12345678
Ad: Alper
Soyad: Çukurova
Öğrenci No: 221220079
Bakiye: 18.00 ₺

Şifre Güncelle

Mevcut Şifre

Yeni Şifre

Yeni Şifre (Tekrar)

Şifreyi Güncelle

- **Bakiye Yükleme Ekranı (Bakiye Yükle/Balance Top-Up):** Kullanıcıların kartlarına bakiye yükleyebildiği ekrandır. Kart numarası, CVV, son kullanım tarihi ve yüklenecek tutar bilgileri girilerek bakiye yükleme işlemi yapılır.

Bakiye Yükleme

Bakiye Yükle

← Bakiye

Bakiye Yükle

YÜKLE

- **Hesap Hareketleri Ekranı (Hesap Hareketleri / Transaction History):** Kullanıcıların geçmişte yaptığı tüm işlemleri (alışveriş, bakiye yükleme vb.) listeleyen ekrandır. İşlem türü, tarih, tutar ve kalan bakiye gibi bilgiler detaylı şekilde gösterilir. Filtreleme ve detay görüntüleme seçenekleri de bulunur.

Hesap Hareketleri

İşlem Filtresi:

Son 10 İşlem

Tarih	İşlem Türü	Tutar	Kalan Bakiye	
2025-05-28 17:43	Alışveriş	-₺9.00	₺18.00	<button>Detayı Gizle</button>
Alışveriş: Su (500ml) (3 adet)				
2025-05-28 17:42	Alışveriş	-₺10.00	₺27.00	<button>Detayı Göster</button>
2025-05-28 17:42	Alışveriş	-₺63.00	₺37.00	<button>Detayı Göster</button>
2025-05-28 17:41	Bakiye Yükleme	+₺100.00	₺100.00	

← HesapHareketleri

İşlem Geçmişi

İşlem Filtresi:

Son 10 İşlem

Tarih: 2025-05-28 17:43
İşlem: Alışveriş
-₺9.00
Kalan Bakiye: ₺18.00

İşlem Detayı

Tarih: 2025-05-28 17:42
İşlem: Alışveriş
-₺10.00
Kalan Bakiye: ₺27.00

İşlem Detayı

Tarih: 2025-05-28 17:42
İşlem: Alışveriş
-₺63.00
Kalan Bakiye: ₺37.00

İşlem Detayı

- **Kart Kayboldu Ekranı (Kart Kayboldu/Lost Card):**
Kullanıcıların mevcut kartlarını dondurup yeni kart talep edebildiği ekrandır. Güvenlik için şifre doğrulaması istenir. Hem web hem mobilde aynı işlevi görür.

Kart Kayboldu

Önce mevcut kartınızı dondurun, ardından yeni kartınızı tanımlayın.

Şifre:

Kartı Dondur

← KartKayboldu

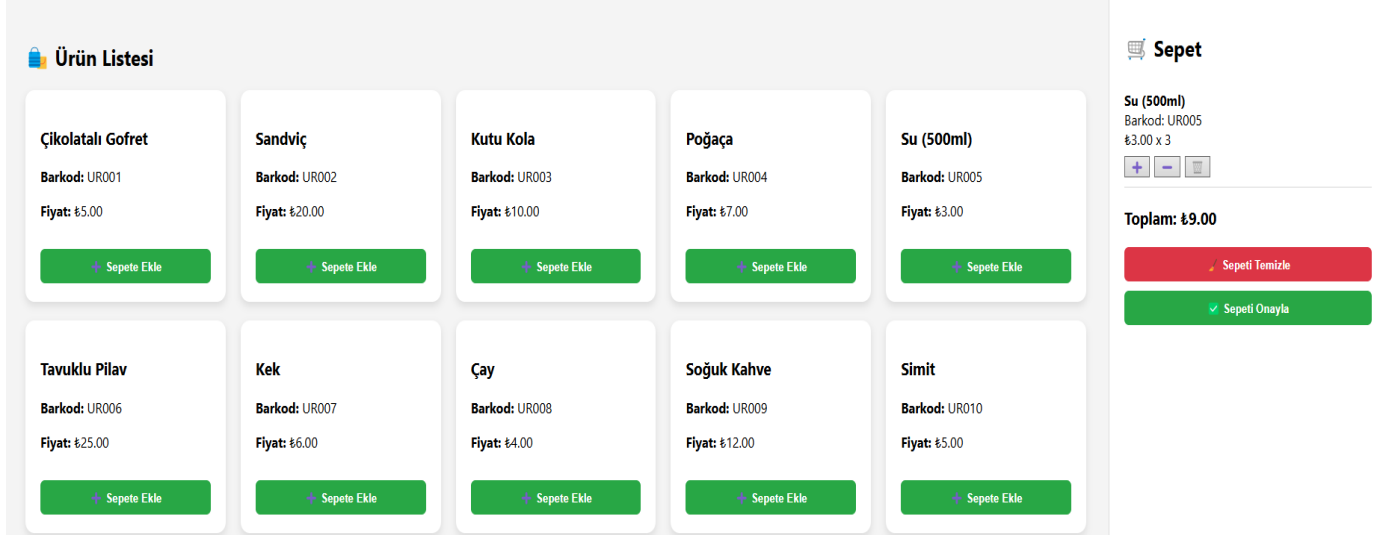
Kartım Kayboldu

Önce mevcut kartınızı dondurun, ardından yeni kartınızı tanımlayın.

Şifre

Kartı Dondur

- **Market Ekranı**



Öğrencilerin kantinden alışveriş yapabilmesini sağlayan kullanıcı arayüzü var. Ürün listesi görüntüleme ve sepet yönetimi, kart ile ödeme işlemlerini gerçekleştirme işlevi vardır.

- **Bileşen Yapısı**

React hooks kullanılarak modern state yönetimi
Context API ile sepet yönetimi
Axios ile HTTP istekleri

- **Kullanıcı Arayüzü Bölümleri**

Ürün Listesi Bölümü
Grid layout ile responsive tasarım
Her ürün için:
Ürün adı
Barkod bilgisi
Fiyat bilgisi
"Sepete Ekle" butonu

- **Sepet Paneli**

Sabit konumlu (sticky) tasarım
Sepet içeriği:
Ürün listesi
Miktar kontrol butonları
Toplam fiyat gösterimi
İşlem butonları:
Sepeti Temizle
Sepeti Onayla
Gerçek zamanlı durum mesajları

- Backend API Kodları
 - Kayıt Ol (Post /register)

```
app.post('/register', async (req, res) => {
  const { kartId, ogrenciNo, firstName, lastName, password } = req.body;

  console.log('Gelen req.body:', req.body);

  if (!ogrenciNo || typeof ogrenciNo !== 'string') {
    return res.status(400).json({ message: 'Geçersiz Öğrenci Numarası' });
  }
  try {
    await sql.connect(config);

    const kartRequest = new sql.Request();
    kartRequest.input('KartId', sql.VarChar(50), kartId.trim());

    const kartResult = await kartRequest.query(
      'SELECT IsAssigned FROM Kartlar WHERE RTRIM(LTRIM(KartID)) = RTRIM(LTRIM(@KartId))'
    );
    if (kartResult.recordset.length === 0) {
      return res.status(400).json({ message: 'Geçersiz kart ID' });
    }
    if (kartResult.recordset[0].IsAssigned === true) {
      return res.status(400).json({ message: 'Bu kart zaten bir kullanıcıya ait' });
    }
    const userRequest = new sql.Request();
    await userRequest
      .input('KartId', sql.VarChar(50), kartId.trim())
      .input('OgrenciNo', sql.VarChar(20), ogrenciNo.trim())
      .input('Ad', sql.VarChar(50), firstName.trim())
      .input('Soyad', sql.VarChar(50), lastName.trim())
      .input('Sifre', sql.VarChar(50), password)
      .query(
        'INSERT INTO Kullanicilar (KartId, OgrenciNo, Ad, Soyad, Sifre) VALUES (@KartId, @OgrenciNo, @Ad, @Soyad, @Sifre)'
      );
    const updateRequest = new sql.Request();
    await updateRequest
      .input('KartId', sql.VarChar(50), kartId.trim())
      .query('UPDATE Kartlar SET IsAssigned = 1 WHERE RTRIM(LTRIM(KartID)) = RTRIM(LTRIM(@KartId))');
    res.status(200).json({ message: 'Kayıt başarıyla' });
  } catch (err) {
    console.error('Hata:', err);
    res.status(500).json({ message: 'Sunucu hatası' });
  }
});
```

Yeni bir kullanıcı (öğrenci) sisteme kaydolur.

Kart ID'si ve öğrenci numarası ile birlikte ad, soyad ve şifre alınır.

Kartın daha önce atanıp atanmadığı kontrol edilir.

Kart uygunsa kullanıcı veritabanına eklenir ve kart atanmış olarak işaretlenir.

- Giriş Yap (Post /login)

```
app.post('/login', async (req, res) => {
  const { ogrenciNo, password } = req.body;

  try {
    await sql.connect(config);
    const request = new sql.Request();
    request.input('OgrenciNo', sql.VarChar(50), ogrenciNo.trim());

    const ogrenciResult = await request.query(
      'SELECT * FROM Kullanicilar WHERE RTRIM(LTRIM(OgrenciNo)) = RTRIM(LTRIM(@OgrenciNo))'
    );
    if (ogrenciResult.recordset.length === 0) {
      return res.status(401).json({ message: 'Öğrenci numarası hatalı', resetPassword: true });
    }
    const user = ogrenciResult.recordset[0];
    if (user.Sifre !== password) {
      return res.status(401).json({ message: 'Şifre hatalı', resetPassword: true });
    }
    res.status(200).json({
      message: 'Giriş başarıyla',
      user: {
        id: user.Id,
        ad: user.Ad,
        soyad: user.Soyad,
        ogrenciNo: user.OgrenciNo,
        kartId: user.KartId,
      }
    });
  } catch (err) {
    console.error('Giriş hatası:', err);
    res.status(500).json({ message: 'Sunucu hatası' });
  }
});
```

Öğrencinin sisteme giriş yapmasını sağlar.
Öğrenci numarası ve şifre alınır.
Kullanıcı veritabanında aranır ve şifre kontrolü yapılır.
Bilgiler doğruysa giriş başarılı olur ve kullanıcı bilgileri döner.

- Profil (Get /profil)

```
app.get('/profil', async (req, res) => {
  const ogrenciNo = req.query.ogrenciNo;
  try {
    await sql.connect(config);

    const request = new sql.Request();
    request.input('OgrenciNo', sql.VarChar(50), ogrenciNo.trim());

    const result = await request.query(`
      SELECT KartId, OgrenciNo, Ad, Soyad, Bakiye, KayitTarihi
      FROM Kullanicilar
      WHERE OgrenciNo = @OgrenciNo
    `);

    if (result.recordset.length === 0) {
      return res.status(404).json({ message: 'Kullanıcı bulunamadı' });
    }

    res.status(200).json(result.recordset[0]);
  } catch (err) {
    console.error('Profil hatası:', err);
    res.status(500).json({ message: 'Sunucu hatası' });
  }
});
```

Öğrencinin profil bilgilerini (kart ID, ad, soyad, bakiye, kayıt tarihi) getirir.
Sorgu parametresi olarak öğrenci numarası alınır.
İlgili öğrenci veritabanından çekilir ve bilgileri döndürülür.

- **Bakiye Yükleme (Post /update-balance)**

```
app.post('/update-balance', async (req, res) => {
  const { ogrenciNo, yeniBakiye } = req.body;
  try {
    await sql.connect(config);
    const getRequest = new sql.Request();
    getRequest.input('OgrenciNo', sql.VarChar(50), ogrenciNo.trim());
    const result = await getRequest.query(`
      SELECT Bakiye FROM Kullanicilar
      WHERE OgrenciNo = @OgrenciNo
    `);
    if (result.recordset.length === 0) {
      return res.status(404).json({ message: 'Kullanıcı bulunamadı.' });
    }
    const mevcutBakiye = parseFloat(result.recordset[0].Bakiye || 0);
    const eklenecekBakiye = parseFloat(yeniBakiye);
    const toplamBakiye = mevcutBakiye + eklenecekBakiye;
    const updateRequest = new sql.Request();
    updateRequest.input('OgrenciNo', sql.VarChar(50), ogrenciNo.trim());
    updateRequest.input('ToplamBakiye', sql.Decimal(10, 2), toplamBakiye);

    await updateRequest.query(`
      UPDATE Kullanicilar
      SET Bakiye = @ToplamBakiye
      WHERE OgrenciNo = @OgrenciNo
    `);

    const hareketRequest = new sql.Request();
    hareketRequest.input('OgrenciNo', sql.VarChar(50), ogrenciNo.trim());
    hareketRequest.input('IslemTuru', sql.VarChar(50), 'Bakiye Yükleme');
    hareketRequest.input('Tutar', sql.Decimal(10, 2), eklenecekBakiye);
    hareketRequest.input('KalanBakiye', sql.Decimal(10, 2), toplamBakiye);
    await hareketRequest.query(`
      INSERT INTO HesapHareketleri (ogrenciNo, islemTuru, tutar, bakiye, islemTarihi)
      VALUES (@OgrenciNo, @IslemTuru, @Tutar, @KalanBakiye, GETDATE())
    `);
    res.status(200).json({
      message: 'Bakiye başarıyla eklendi ve hesap hareketine kaydedildi.',
      toplamBakiye: toplamBakiye.toFixed(2)
    });
  } catch (err) {
    console.error('Bakiye güncelleme hatası:', err);
    res.status(500).json({ message: 'Sunucu hatası' });
  }
});
```

Öğrencinin bakiyesini günceller ve bu işlemi hareketler tablosuna kaydeder.

Öğrenci numarası ve yüklenecek bakiye alınır.

Mevcut bakiye ile toplanır ve güncellenir.

İşlem hareketler tablosuna kaydedilir.

- **Hesap Hareketleri (Get /hesap-hareketleri)**

```
app.get('/hesap-hareketleri', async (req, res) => {
  const { ogrenciNo } = req.query;

  try {
    await sql.connect(config);

    const request = new sql.Request();
    request.input('OgrenciNo', sql.VarChar(50), ogrenciNo.trim());

    const result = await request.query(`
      SELECT
        islemTarihi AS Tarih,
        islemTuru AS IslemTuru,
        tutar AS Tutar,
        bakiye AS KalanBakiye
      FROM HesapHareketleri
      WHERE ogrenciNo = @OgrenciNo
      ORDER BY islemTarihi DESC
    `);

    res.status(200).json(result.recordset);
  } catch (err) {
    console.error('Hesap hareketleri hatası:', err);
    res.status(500).json({ message: 'Sunucu hatası' });
  }
});
```

Öğrencinin tüm hesap hareketlerini (alışveriş, bakiye yükleme vb.) listeler.
Sorgu parametresi olarak öğrenci numarası alınır.
İlgili öğrenciye ait tüm hareketler, tarih sırasına göre döndürülür.

- **Şifre Güncelle (Post /update-password)**

```
app.post('/update-password', async (req, res) => {
  const { ogrenciNo, currentPassword, newPassword } = req.body;

  try {
    await sql.connect(config);
    const checkRequest = new sql.Request();
    checkRequest.input('OgrenciNo', sql.VarChar(50), ogrenciNo.trim());
    const userResult = await checkRequest.query(`
      SELECT Sifre FROM Kullanicilar
      WHERE RTRIM(LTRIM(OgrenciNo)) = RTRIM(LTRIM(@OgrenciNo))
    `);
    if (userResult.recordset.length === 0) {
      return res.status(404).json({ message: 'Kullanıcı bulunamadı' });
    }
    if (userResult.recordset[0].Sifre !== currentPassword) {
      return res.status(401).json({ message: 'Mevcut şifre hatalı' });
    }
    const updateRequest = new sql.Request();
    updateRequest
      .input('OgrenciNo', sql.VarChar(50), ogrenciNo.trim())
      .input('YeniSifre', sql.VarChar(50), newPassword);
    await updateRequest.query(`
      UPDATE Kullanicilar
      SET Sifre = @YeniSifre
      WHERE RTRIM(LTRIM(OgrenciNo)) = RTRIM(LTRIM(@OgrenciNo))
    `);
    res.status(200).json({ message: 'Şifre başarıyla güncellendi.' });
  } catch (err) {
    console.error('Şifre güncelleme hatası:', err);
    res.status(500).json({ message: 'Sunucu hatası' });
  }
});

app.listen(port, () => {
  console.log(`Server çalışıyor: http://localhost:${port}`);
});
```

Öğrencinin şifresini günceller.
Öğrenci numarası, mevcut şifre ve yeni şifre alınır.
Mevcut şifre doğruysa yeni şifre ile güncellenir.

- **Kart Kayboldu (Post /kayip-kart)**

```
app.post('/kayip-kart', async (req, res) => {
  const { ogrenciNo, sifre, yeniKartId, adim } = req.body;

  try {
    await sql.connect(config);

    const userRequest = new sql.Request();
    userRequest.input('OgrenciNo', sql.VarChar(50), ogrenciNo.trim());
    const userResult = await userRequest.query(`
      SELECT * FROM Kullanicilar
      WHERE RTRIM(LTRIM(OgrenciNo)) = RTRIM(LTRIM(@OgrenciNo))
    `);
    if (userResult.recordset.length === 0) {
      return res.status(404).json({ message: 'Kullanıcı bulunamadı' });
    }
    const user = userResult.recordset[0];
    if (user.Sifre !== sifre) {
      return res.status(401).json({ message: 'Şifre hatalı' });
    }

    if (adim === "dondur") {
      const freezeRequest = new sql.Request();
      freezeRequest.input('KartId', sql.VarChar(50), user.KartId.trim());

      const freezeResult = await freezeRequest.query(`
        UPDATE Kartlar
        SET IsFrozen = 1
        WHERE LOWER(RTRIM(LTRIM(KartID))) = LOWER(RTRIM(LTRIM(@KartId)))
      `);
      console.log("Kart donduruldu. Güncellenen satır:", freezeResult.rowsAffected);
      return res.status(200).json({ message: 'Kart başarıyla donduruldu.' });
    }

    if (adim === "degistir") {
      const kartRequest = new sql.Request();
      kartRequest.input('KartId', sql.VarChar(50), yeniKartId.trim());
      const kartResult = await kartRequest.query(`
        SELECT IsAssigned FROM Kartlar WHERE RTRIM(LTRIM(KartID)) = RTRIM(LTRIM(@KartId))
      `);
      if (kartResult.recordset.length === 0) {
        return res.status(400).json({ message: 'Yeni kart ID geçersiz' });
      }
      if (kartResult.recordset[0].IsAssigned === true) {
        return res.status(400).json({ message: 'Yeni kart başka bir kullanıcıya ait' });
      }
      const eskiKartRequest = new sql.Request();
      eskiKartRequest.input('KartId', sql.VarChar(50), user.KartId.trim());
      const eskiKartGuncelle = await eskiKartRequest.query(`
        UPDATE Kartlar
        SET IsAssigned = 1, IsFrozen = 1
        WHERE LOWER(RTRIM(LTRIM(KartID))) = LOWER(RTRIM(LTRIM(@KartId)))
      `);
      console.log("Eski kart güncellendi:", eskiKartGuncelle.rowsAffected);

      const yeniKartRequest = new sql.Request();
      yeniKartRequest.input('KartId', sql.VarChar(50), yeniKartId.trim());
      await yeniKartRequest.query(`
        UPDATE Kartlar SET IsAssigned = 1, IsFrozen = 0 WHERE RTRIM(LTRIM(KartID)) = RTRIM(LTRIM(@KartId))
      `);
      const updateUserRequest = new sql.Request();
      updateUserRequest
        .input('YeniKartId', sql.VarChar(50), yeniKartId.trim())
        .input('OgrenciNo', sql.VarChar(50), ogrenciNo.trim());
      await updateUserRequest.query(`
        UPDATE Kullanicilar
        SET KartId = @YeniKartId
        WHERE OgrenciNo = @OgrenciNo
      `);
      return res.status(200).json({ message: 'Yeni kart başarıyla atandı, bilgiler korundu.' });
    }

    // Adım yanlışsa TAB to jump here
    res.status(400).json({ message: 'Geçersiz işlem adı' });
  } catch (err) {
    console.error('Kart işlemi hatası:', err);
    res.status(500).json({ message: 'Sunucu hatası' });
  }
});
```

Öğrencinin kartı kaybolduğunda önce mevcut kartı dondurmak, ardından yeni bir kart tanımlamak için kullanılır.

adim parametresi "dondur" ise mevcut kart dondurulur.

adim parametresi "degistir" ise yeni kart atanır ve eski kart dondurulmuş olarak işaretlenir.

Şifre doğrulaması yapılır.

- **Urunler (Get /urunler)**

```
app.get('/urunler', async (req, res) => {
  try {
    await sql.connect(config);
    const result = await new sql.Request().query('SELECT * FROM Urunler');
    res.status(200).json(result.recordset);
  } catch (err) {
    console.error('Ürünler alınamadı:', err);
    res.status(500).json({ message: 'Ürünleri çekerken hata oluştu' });
  }
});
```

Tüm ürünleri veritabanından çeker

Hata yönetimi ve uygun HTTP durum kodları kullanılmıştır

CORS desteği ile güvenli cross-origin istekleri sağlanmıştır

- **Kart Okuma (Post /api/kart-okuma)**

```
app.post('/api/kart-okuma', (req, res) => {
  const kartId = req.body.kartId;
  if (!kartId) {
    return res.status(400).json({ success: false, message: 'Kart ID eksik' });
  }
  bekleyenKartId = kartId;
  console.log('Yeni kart okundu:', kartId);
  res.json({ success: true });
});
```

Arduino'dan gelen kart ID'sini alır

Geçici olarak kart ID'sini saklar

Hata kontrolü ile geçersiz istekleri reddeder

- **Kart Okuma (Get /api/kart-okuma)**

```
app.get('/api/kart-okuma', (req, res) => {
  if (bekleyenKartId) {
    const kartId = bekleyenKartId;
    bekleyenKartId = null;
    res.json({ kartId });
  } else {
    res.json({ kartId: null });
  }
});
```

Frontend'in kart ID'sini sorgulamasını sağlar

Okunan kart ID'sini döndürür ve sıfırlar

Asenkron iletişim için polling mekanizması sağlar

- Sepet Onayı (Post /api/kullanici/kartGeldi)

```
app.post('/api/kullanici/kartGeldi', async (req, res) => {
  const kartId = req.body.kartId;
  const sepetTutari = req.body.sepetTutari;
  const urunler = req.body.urunler || [];

  if (!kartId || !sepetTutari || urunler.length === 0) {
    return res.status(400).json({ success: false, message: 'Eksik veri: kartId, sepetTutari veya urunler.' });
  }

  try {
    await sql.connect(config);
    const result = await new sql.Request()
      .input('kartId', sql.NVarChar, kartId)
      .query('SELECT ogrenciNo, bakiye FROM Kullanicilar WHERE kartId = @kartId');
    if (result.recordset.length === 0) {
      return res.status(404).json({ success: false, message: 'Kullanıcı bulunamadı.' });
    }
    const { ogrenciNo, bakiye } = result.recordset[0];
    if (bakiye < sepetTutari) {
      return res.status(400).json({ success: false, message: 'Bakiye yetersiz.' });
    }
    const yeniBakiye = bakiye - sepetTutari;
    await new sql.Request()
      .input('ogrenciNo', sql.NVarChar, ogrenciNo)
      .input('sepetTutari', sql.Decimal(10, 2), sepetTutari)
      .query('UPDATE Kullanicilar SET bakiye = bakiye - @sepetTutari WHERE ogrenciNo = @ogrenciNo');

    const urunKodlari = urunler.map(u => `${u.urunKodu}`).join(',');
    const urunQuery = `SELECT urunKodu, urunAdi FROM Urunler WHERE urunKodu IN (${urunKodlari})`;
    const urunResult = await new sql.Request().query(urunQuery);
    const urunMap = {};
    urunResult.recordset.forEach(u => { urunMap[u.urunKodu] = u.urunAdi; });

    const urunIsimleri = urunler.map(urun => {
      const ad = urunMap[urun.urunKodu] || 'Bilinmeyen Ürün';
      return `${ad} (${urun.adet} adet)`;
    });

    const islemDetayi = `Alışveriş: ${urunIsimleri.join(', ')}`;
    const hareketResult = await new sql.Request()
      .input('ogrenciNo', sql.NVarChar, ogrenciNo)
      .input('islemTuru', sql.NVarChar, islemDetayi)
      .input('tutar', sql.Decimal(10, 2), -sepetTutari)
      .input('bakiye', sql.Decimal(10, 2), yeniBakiye)
      .query(`
        INSERT INTO HesapHareketleri (ogrenciNo, islemTuru, tutar, bakiye, islemTarihi)
        OUTPUT INSERTED.id
        VALUES (@ogrenciNo, @islemTuru, @tutar, @bakiye, GETDATE())
      `);

    const hareketId = hareketResult.recordset[0].id;
    for (const urun of urunler) {
      await new sql.Request()
        .input('hareketId', sql.Int, hareketId)
        .input('urunKodu', sql.NVarChar, urun.urunKodu)
        .input('adet', sql.Int, urun.adet)
        .query(`
          INSERT INTO SatinAlinanUrunler (hareketId, urunKodu, adet)
          VALUES (@hareketId, @urunKodu, @adet)
        `);
    }
    res.json({ success: true, message: 'Satın alma başarılı.', kalanBakiye: yeniBakiye });
  } catch (err) {
    console.error('Sepet onay hatası:', err);
    res.status(500).json({ success: false, message: 'Sunucu hatası.' });
  }
});
```

Karmaşık işlem mantığı içerir:

Kullanıcı doğrulama

Bakiye kontrolü

Bakiye güncelleme

İşlem kaydı oluşturma

Satın alınan ürünlerin kaydı

Transaction güvenliği sağlanmıştır

Detaylı hata yönetimi içerir

- Veritabanı Bağlantı Yapılandırması

1. Temel Yapı:

```
const config = {
  server: 'ALPEREN-MSI\\alpcu',
  database: 'KantinDB',
  options: {
    encrypt: true,
    trustServerCertificate: true
  },
  connectionString: "Driver={ODBC Driver 17 for SQL Server};Server=ALPEREN-MSI\\SQLEXPRESS;Database=KantinDB;Trusted_Connection=Yes;",
  driver: "msnodesqlv8"
};
```

2. Bağlantı Detayları:

- Sunucu Bilgisi: ALPEREN-MSI\\alpcu
- Yerel bilgisayar adı ve SQL Server instance'ı
- Yerel geliştirme ortamında çalışıyor
- Veritabanı: KantinDB
- Kampüs kantin sistemi için özel oluşturulmuş veritabanı
- Tüm tablolar ve veriler burada saklanıyor

3. Güvenlik Özellikleri:

- Şifreleme: encrypt: true
- Veri transferi sırasında şifreleme aktif
- Hassas bilgilerin güvenliği sağlanıyor
- Sertifika Doğrulama: trustServerCertificate: true
- SSL sertifikası kontrolü
- Güvenli bağlantı için gerekli

4. Bağlantı Yöntemi:

- ODBC Driver: ODBC Driver 17 for SQL Server
- En güncel SQL Server sürücüsü
- Modern özellikleri destekliyor
- Kimlik Doğrulama: Trusted_Connection=Yes
- Windows Authentication kullanılıyor
- Kullanıcı adı/şifre yerine Windows kimlik bilgileri
- Daha güvenli ve yönetimi kolay

Projede tüm veritabanı işlemleri, merkezi bir yapılandırma üzerinden yönetilmektedir. Bu yapılandırma, temel yapı içinde tanımlanmış olup, tüm API endpoint'leri tarafından kullanılmaktadır.

Bu merkezi yapılandırma sayesinde, tüm veritabanı işlemleri aynı güvenlik ve performans standartlarında gerçekleştirilmektedir. Herhangi bir API endpoint'i veritabanına erişmek istediğinde, bu yapılandırmayı kullanarak güvenli bir bağlantı kurar ve işlemlerini gerçekleştirir.

- VeriTabanı Açıklamaları

- Kullanıcılar Tablosu

```
-- Kullanıcılar
CREATE TABLE Kullanıcılar (
    KartId VARCHAR(50),
    OgrenciNo VARCHAR(20) PRIMARY KEY,
    Ad VARCHAR(50),
    Soyad VARCHAR(50),
    Sifre VARCHAR(50),
    Bakiye DECIMAL(10,2),
    KayitTarihi DATETIME
);
```

Amaç: Sistemdeki tüm öğrencilerin kimlik, kart ve bakiye bilgilerini tutar.

Temel Alanlar:

OgrenciNo (PK)

KartId (her öğrenciye atanmış kart)

Ad, Soyad, Sifre, Bakiye, KayitTarihi

- Kartlar Tablosu

```
CREATE TABLE Kartlar (
    KartId VARCHAR(50) PRIMARY KEY,
    IsAssigned BIT,
    IsFrozen BIT
);
```

Amaç: Sistemdeki tüm kartların durumunu ve atanıp atanmadığını tutar.

Temel Alanlar:

KartId (PK)

IsAssigned (kart atanmış mı)

IsFrozen (kart dondurulmuş mu)

- Hesap Hareketleri Tablosu

```
CREATE TABLE HesapHareketleri (
    id INT IDENTITY(1,1) PRIMARY KEY,
    OgrenciNo VARCHAR(20),
    IslemTarihi DATETIME,
    IslemTuru VARCHAR(100),
    Tutar DECIMAL(10,2),
    Bakiye DECIMAL(10,2),
    FOREIGN KEY (OgrenciNo) REFERENCES Kullanıcılar(OgrenciNo)
);
```

Amaç: Her öğrencinin yaptığı tüm finansal işlemleri (alışveriş, bakiye yükleme vb.) kaydeder.

Temel Alanlar:

id (PK)

OgrenciNo (Kullanıcılar tablosuna bağlı)

IslemTarihi, IslemTuru, Tutar, Bakiye

- **Urunler Tablosu**

```
CREATE TABLE Urunler (  
    id INT IDENTITY(1,1) PRIMARY KEY,  
    urunKodu VARCHAR(20) NOT NULL UNIQUE,  
    urunAdi VARCHAR(100) NOT NULL,  
    fiyat DECIMAL(10,2) NOT NULL  
);
```

Amaç: Kantinde satılan tüm ürünlerin bilgisini tutar.

Temel Alanlar:

id (PK)

urunKodu (benzersiz, UNIQUE)

urunAdi, fiyat

- **SatinAlinanUrunler Tablosu**

```
CREATE TABLE SatinAlinanUrunler (  
    id INT IDENTITY(1,1) PRIMARY KEY,  
    hareketId INT NOT NULL,  
    urunKodu VARCHAR(20) NOT NULL,  
    adet INT NOT NULL,  
    FOREIGN KEY (hareketId) REFERENCES HesapHareketleri(id)  
    -- urunKodu için de isterseniz FOREIGN KEY (urunKodu) REFERENCES Urunler(urunKodu) ekleyebilirsiniz  
);
```

Amaç: Her alışverişte hangi ürünlerin, hangi miktarda alındığını kaydeder.

Temel Alanlar:

id (PK)

hareketId (HesapHareketleri tablosuna bağlı, FK)

urunKodu (Urunler tablosuna bağlı, mantıksal FK)

adet

TABLolar ARASI İLİŞKİLER

- Kullanıcılar ile Kartlar arasında bire-bir ilişki vardır. Her öğrenciye bir kart atanır.
- HesapHareketleri tablosunda, her işlem bir öğrenciye (OgrenciNo) aittir. Yani Kullanıcılar ile HesapHareketleri arasında bire-çok (1-N) ilişki vardır.
- SatinAlinanUrunler tablosunda, her kayıt bir HesapHareketleri kaydına (hareketId) bağlıdır. Yani bir işlemde birden fazla ürün alınabilir (1-N).
- SatinAlinanUrunler tablosundaki urunKodu, Urunler tablosundaki urunKodu ile eşleşir. Böylece hangi ürünün alındığı belirlenir.

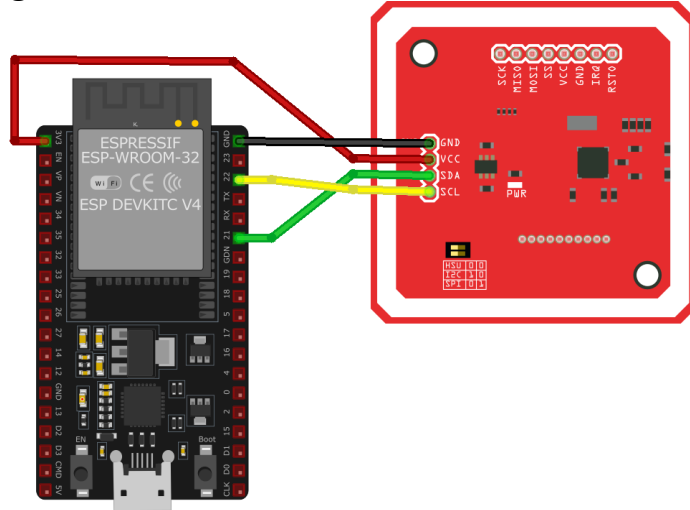
○ ESP32 PN532 I2C Bağlantısı

• ESP32 ve PN532 Nedir?

ESP32: WiFi ve Bluetooth özelliklerine sahip, düşük maliyetli ve güçlü bir mikrodenetleyicidir. Projende, kart okuyucu (PN532) ile haberleşip okunan kart bilgisini WiFi üzerinden sunucuya göndermek için kullanılır.

PN532: NFC (yakın alan iletişimi) ve RFID kartlarını okuyabilen bir modüldür. Öğrencilerin kartlarını okutmak için kullanılır. ESP32 ile haberleşerek kartın ID'sini okur ve bu bilgiyi ESP32'ye iletir.

• I2C Bağlantısı Nedir ve Nasıl Kullanıldı?



fritzing

```
void setup() {  
  Serial.begin(115200);  
  Wire.begin(21, 22);  
  Wire.setClock(10000);  
}
```

I2C (Inter-Integrated Circuit): Birden fazla cihazın (örneğin ESP32 ve PN532) iki kablo (SDA ve SCL) üzerinden haberleşmesini sağlayan bir protokoldür.

SDA: Veri hattı

SCL: Saat hattı

Projende, ESP32 ile PN532 arasında I2C bağlantısı kurularak hızlı ve güvenli veri aktarımı sağlanır. Kodda Wire.begin(21, 22); satırı ile ESP32'nin 21 numaralı pini SDA, 22 numaralı pini ise SCL olarak ayarlanmıştır.

- **ESP32 Kodunun setup() Açıklaması**

```
const char* ssid = "Alperen";
const char* password = "1453alpern";

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
  Wire.setClock(10000); // Hız artırıldı

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("WiFi'ya bağlanıyor...");
  }
  Serial.println("WiFi bağlantısı başarılı!");

  Wire.beginTransmission(PN532_I2C_ADDR);
  if (Wire.endTransmission() != 0) {
    Serial.println("I2C cihazına bağlanılamadı!");
    while (1);
  }

  nfc.begin();
  if (!nfc.getFirmwareVersion()) {
    Serial.println("PN532 modülü bulunamadı!");
    while (1);
  }

  nfc.SAMConfig();
  Serial.println("Kart bekleniyor...");
}
```

setup() Fonksiyonu:

Amaç: Donanım ve bağlantıların ilk ayarlarını yapar, sistemi başlatır.

Serial.begin(115200);

Seri portu başlatır, hata ayıklama ve bilgi mesajları için kullanılır.

Wire.begin(21, 22);

I2C haberleşmesini başlatır (SDA: 21, SCL: 22).

WiFi.begin(ssid, password);

ESP32'yi belirtilen WiFi ağına bağlar.

WiFi bağlantısı kontrolü:

Bağlantı sağlanana kadar bekler ve kullanıcıya bilgi verir.

I2C cihazı ve PN532 modülü kontrolü:

PN532 modülünün bağlı ve çalışır olduğundan emin olunur.

Bağlantı veya modül hatası varsa sistem durur.

nfc.SAMConfig();

PN532'yi kart okuma moduna alır.

Serial.println("\nKart bekleniyor...\n");

Kullanıcıya sistemin kart beklediğini bildirir.

- **ESP32 Kodunun loop() Açıklaması**

```
void loop() {
    uint8_t uid[7];
    uint8_t uidLength;

    if (nfc.readPassiveTargetID(PN532_MIFARE_ISO14443A, uid, &uidLength)) {
        Serial.println("Kart bulundu!");

        String cardId = "";
        for (uint8_t i = 0; i < uidLength; i++) {
            if (uid[i] < 0x10) cardId += "0";
            cardId += String(uid[i], HEX);
        }
        cardId.toLowerCase();

        Serial.print("Kart ID: "); Serial.println(cardId);

        if (WiFi.status() == WL_CONNECTED) {
            HTTPClient http;
            http.begin("http://192.168.110.56:3003/api/kart-okuma"); // Sunucu IP
            http.addHeader("Content-Type", "application/json");

            String payload = "{\"kartId\":\"" + cardId + "\"}";
            int httpCode = http.POST(payload);
            if (httpCode > 0) {
                String payload = http.getString();
                Serial.println("Kart ID başarıyla gönderildi.");
                Serial.println(payload);
            } else {
                Serial.print("HTTP Hatası: ");
                Serial.println(httpCode);
                Serial.println(http.errorToString(httpCode)); // EKLE
            }

            http.end();
        } else {
            Serial.println("WiFi bağlantısı yok!");
        }

        delay(3000); // Aynı kartın tekrar okutulmasını engellemek için bekleme
    }
}
```

Amaç: Sürekli olarak kart okutulup okutulmadığını kontrol eder ve okunan kart ID'sini sunucuya gönderir.

Kart Okuma:

nfc.readPassiveTargetID(...) fonksiyonu ile bir kart okutulup okutulmadığı kontrol edilir.

Kart Okunduysa:

Kartın UID'si (benzersiz ID) alınır ve string olarak birleştirilir.

Kart ID'si seri port üzerinden ekrana yazdırılır.

WiFi Bağlantısı Kontrolü:

WiFi bağlıysa, HTTPClient ile sunucuya (API'ye) POST isteği gönderilir.

JSON formatında kart ID'si sunucuya iletilir.

Sunucudan gelen yanıt ekrana yazdırılır.

Hata olursa hata kodu ve açıklaması ekrana yazdırılır.

WiFi Yoksa:

“WiFi bağlantısı yok!” mesajı ekrana yazdırılır.

delay(3000);

Aynı kartın tekrar okutulmasını engellemek için 3 saniye beklenir.

MATERYAL VE METOT

1. Kullanılan Donanımlar

- ESP32 Geliştirme Kartı:
Projenin ana mikrodenetleyicisi olarak kullanılmıştır. Hem WiFi bağlantısı hem de hızlı işlem gücü sayesinde, kart okuma ve sunucuya veri gönderme işlemlerini üstlenir.
- PN532 NFC Kart Okuyucu:
Öğrencilerin kartlarını okutmak için kullanılmıştır. Kartın benzersiz ID'sini okur ve ESP32'ye iletir.
- Bağlantı Kabloları ve Breadboard:
ESP32 ile PN532 arasında I2C protokolüyle bağlantı kurulmuştur. Gerekli bağlantılar breadboard veya doğrudan lehimleme ile sağlanmıştır.
- Güç Kaynağı (USB veya Pil):
ESP32 ve PN532 modülünün çalışması için gerekli enerjiyi sağlar.

2. Kullanılan Yazılımlar ve Teknolojiler

- Arduino IDE & PlatformIO:
ESP32 üzerinde gömülü yazılım geliştirmek için kullanılmıştır.
- C/C++ Programlama Dili:
ESP32 ve PN532 arasındaki donanım haberleşmesi ve WiFi üzerinden veri gönderimi için kullanılmıştır.
- Node.js (Express.js):
Backend API'lerinin geliştirilmesinde kullanılmıştır. Tüm veri işleme, kullanıcı yönetimi ve kart kontrolü işlemleri burada yapılır.
- Microsoft SQL Server (MSSQL):
Tüm kullanıcı, kart, ürün ve işlem verilerinin saklandığı ilişkisel veritabanı olarak kullanılmıştır.
- React & React Native:
Web ve mobil uygulama arayüzlerinin geliştirilmesinde kullanılmıştır. Kullanıcılar, bakiye sorgulama, işlem geçmişi görüntüleme ve kart yönetimi gibi işlemleri bu arayüzler üzerinden yapar.
- Axios:
Web ve mobil uygulamalarda backend API'lerine HTTP istekleri göndermek için kullanılmıştır.

3. Donanım Bağlantı Şeması ve Montaj

- ESP32 ile PN532 Arası Bağlantı:
PN532 modülü, I2C protokolü ile ESP32'nin GPIO21 (SDA) ve GPIO22 (SCL) pinlerine bağlanmıştır. Bağlantı şu şekildedir:
- PN532 SDA → ESP32 GPIO21
- PN532 SCL → ESP32 GPIO22
- VCC ve GND hatları da uygun şekilde bağlanmıştır.
- Montaj:
Güç kaynağı olarak USB

4. Yazılım Algoritması ve Veri İşleme

- ESP32 ve PN532:
- ESP32, sürekli olarak PN532 üzerinden kart okutulup okutulmadığını kontrol eder.
- Bir kart okutulduğunda, kartın benzersiz ID'si alınır.
- ESP32, WiFi üzerinden bu kart ID'sini backend API'sine gönderir.
- Backend, kartın geçerliliğini ve kullanıcının bakiyesini kontrol eder, sonucu ESP32'ye döner.
- Sonuca göre (örneğin bakiye yeterliyse) işlem onaylanır veya reddedilir.
- Backend (Node.js):
- Kart kontrolü, kullanıcı doğrulama, bakiye güncelleme, ürün satışı ve işlem kaydı gibi tüm işlemler merkezi olarak yönetilir.
- Tüm işlemler MSSQL veritabanında saklanır.

5. Mobil Uygulama Entegrasyonu

- React Native ile Geliştirildi:
Öğrenciler ve veliler için mobil uygulama arayüzü sunar.
- Temel Özellikler:
- Giriş ve kayıt işlemleri
- Profil ve bakiye görüntüleme
- Bakiye yükleme
- Hesap hareketleri ve alışveriş geçmişi
- Kart kayboldu bildirimi ve yeni kart talebi
- API Entegrasyonu:
Mobil uygulama, Axios ile backend API'lerine bağlanarak tüm işlemleri gerçek zamanlı olarak gerçekleştirir.

6. Web Sitesi Entegrasyonu

- React ile Geliştirildi:
Veliler ve öğrenciler için web tabanlı arayüz sunar.
- Temel Özellikler:
- Öğrenci ve kart yönetimi
- Bakiye yükleme ve bakiye geçmişi
- Hesap hareketleri ve detaylı raporlama
- Kart kayboldu işlemleri
- API Entegrasyonu:
Web sitesi, backend API'leri ile entegre çalışır ve tüm veriler merkezi olarak yönetilir.

KAYNAKLAR

Direnç.net. (2024). PN532 NFC Kart Okuyucu Modülü.
<https://www.direnc.net/pn532-nfc-rfid-modulu>

Robotistan Blog. (2023). ESP32 ile NFC Kart Okuma (PN532 Kullanımı).
<https://maker.robotistan.com/esp32-pn532-nfc-kart-okuma/>

Espressif Systems. (2024). ESP32 Teknik Dokümantasyonu.
<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>

Arduino. (2024). Arduino IDE Kullanım Kılavuzu.
<https://www.arduino.cc/en/Guide>

Adafruit. (2024). Adafruit PN532 RFID/NFC Breakout and Shield.
<https://learn.adafruit.com/adafruit-pn532-rfid-nfc>

Microsoft. (2024). SQL Server Documentation.
<https://learn.microsoft.com/en-us/sql/sql-server/>

Patika.dev. (2024). Node.js ile Backend Geliştirme.
<https://www.patika.dev/tr/bootcamp/academy/nodejs>

Patika.dev. (2024). React ve React Native ile Uygulama Geliştirme.
<https://www.patika.dev/tr/bootcamp/academy/react>

React Native. (2024). React Native Resmi Belgeler.
<https://reactnative.dev/docs/getting-started>

Expo. (2024). Expo Go Documentation. <https://docs.expo.dev/>

Axios. (2024). Axios HTTP Client Documentation. <https://axios-http.com/docs/intro>

YouTube. (2023). ESP32 ile Kart Okuma ve Web API Entegrasyonu [Video].
<https://www.youtube.com/watch?v=6Qw1kQw1QwQ>

GitHub. (2024). adafruit/Adafruit-PN532 [Kod deposu].
<https://github.com/adafruit/Adafruit-PN532>

GitHub. (2024). expressjs/express [Kod deposu].
<https://github.com/expressjs/express>